

Լաբորատոր աշխատանք 4

Ծրագրավորման գործնական աշխատանքի նպատակն է յուրացնել և գործնականում կիրառել C++ լեզվի կառուցվածքները (struct), դինամիկ հիշողության կառավարման եղանակները, ինչպես նաև ցուցիչային թվաբանությունը:

Խնդրի պահանջը

Ստեղծել Order կառուցվածք՝ customerName, productName, status և totalPrice դաշտերով:

Մուտքագրել 10 պատվերի տվյալներ: Ֆունկցիաների միջոցով՝ արտածել բոլոր պատվերները,

գտնել ամենաբարձր արժեք ունեցող պատվերը,

արտածել տրված կարգավիճակով պատվերները,

հաշվել բոլոր պատվերների ընդհանուր արժեքը:

Տեսական մաս

Աշխատանքի ավարտին հիշողության արտահոսքից (memory leak) խուսափելու համար դինամիկ հատկացված զանգվածը հեռացվել է delete[] օպերատորով, իսկ ցուցիչները անվտանգության նկատառումներով ստացել են nullptr արժեք:

```
#include <iostream>
```

```
#include <string>
```

```
#include <vector>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
// 1. Պատվերի կառուցվածքը (struct)
```

```
struct Order {  
    string customerName;  
    string productName;  
    string status;  
    double totalPrice;  
};
```

```
// Ֆունկցիա՝ մեկ պատվերի տվյալները տպելու համար
void printOrder(const Order& order) {
    cout << "Customer: " << setw(15) << left << order.customerName
        << " | Product: " << setw(15) << left << order.productName
        << " | Status: " << setw(12) << left << order.status
        << " | Total: $" << fixed << setprecision(2) << order.totalPrice << endl;
}
```

```
// 2. Արտածել բոլոր պատվերները
void printAllOrders(const vector<Order>& orders) {
    cout << "\n=== ALL ORDERS ===" << endl;
    for (size_t i = 0; i < orders.size(); ++i) {
        cout << i + 1 << ". ";
        printOrder(orders[i]);
    }
}
```

```
// 3. Գտնել ամենաբարձր արժեք ունեցող պատվերը
void printMaxOrder(const vector<Order>& orders) {
    if (orders.empty()) return;

    size_t maxIndex = 0;
    for (size_t i = 1; i < orders.size(); ++i) {
        if (orders[i].totalPrice > orders[maxIndex].totalPrice) {
            maxIndex = i;
        }
    }

    cout << "\n=== HIGHEST VALUE ORDER ===" << endl;
    printOrder(orders[maxIndex]);
}
```

```
// 4. Արտածել տրված կարգավիճակով պատվերները
void printOrdersByStatus(const vector<Order>& orders, const string& targetStatus) {
```

```

cout << "\n=== ORDERS WITH STATUS \"" << targetStatus << "\" ===" << endl;
bool found = false;
for (const auto& order : orders) {
    if (order.status == targetStatus) {
        printOrder(order);
        found = true;
    }
}
if (!found) {
    cout << "No orders found with this status." << endl;
}
}

```

// 5. Հաշվել բոլոր պատվերների ընդհանուր արժեքը

```

double calculateTotalSum(const vector<Order>& orders) {
    double totalSum = 0;
    for (const auto& order : orders) {
        totalSum += order.totalPrice;
    }
    return totalSum;
}

```

```

int main() {
    const int ORDER_COUNT = 10;
    vector<Order> orders(ORDER_COUNT);

    cout << "Enter details for " << ORDER_COUNT << " orders:" << endl;
    cout << "-----" << endl;

```

// 10 պատվերների մուտքագրում

```

for (int i = 0; i < ORDER_COUNT; ++i) {
    cout << "\nOrder #" << i + 1 << endl;
    cout << "Customer Name: ";
    cin >> orders[i].customerName;
    cout << "Product Name: ";

```

```

        cin >> orders[i].productName;
        cout << "Status (e.g., Pending, Shipped, Delivered): ";
        cin >> orders[i].status;
        cout << "Total Price: ";
        cin >> orders[i].totalPrice;
    }

    // Ֆունկցիաների կանչեր
    printAllOrders(orders);
    printMaxOrder(orders);

    // Որոնում ըստ կարգավիճակի
    string searchStatus;
    cout << "\nEnter status to filter orders: ";
    cin >> searchStatus;
    printOrdersByStatus(orders, searchStatus);

    // Ընդհանուր գումարի հաշվարկ
    double totalAll = calculateTotalSum(orders);
    cout << "\n===== " << endl;
    cout << "TOTAL VALUE OF ALL ORDERS: $"
        << fixed << setprecision(2) << totalAll << endl;

    return 0;
}

```